Nesne Yönelimli Programlama

Hafta 6: Yapılar, Birlikler ve Numaralandırmalar

Dr. Hüseyin Yüce

Marmara Üniversitesi Teknoloji Fakültesi

Mekatronik Mühendisliği Bölümü

Hafta 6: Teorik ve Kavramsal Hedefler

Bu hafta hedeflenen temel kavramsal kazanımlar:

- Yapılar (`struct`) ve Bileşik Veri Modelleme:
  - Heterojen veri tiplerinin tek bir kullanıcı tanımlı tür altında toplanması
  - Üye erişim operatörleri (`.` ve `->`), yapı dizileri ve iç içe yapılar (*nested structs*)
- Yapılarda Nesne Yönelimli Yetenekler:
  - Yapılarda kurucu (*constructor*), yıkıcı (*destructor*) ve üye ilklendirme listesi kullanımı
  - `struct` ile `class` arasındaki varsayılan erişim ve kalıtım farkları
- Birlikler (`union`) ve Ortak Bellek Paylaşımı:
  - Tek aktif üye prensibi, bayt düzeyinde veri ayrıştırma (*type punning*) ve etiketli birlikler
- Bellek Hizalama, Bit Alanları ve Numaralandırmalar:
  - Doğal bellek hizalama (*alignment*), dolgu baytları (*padding*), `#pragma pack` ve bit alanları (*bit-fields*)
  - Geleneksel `enum` zafiyetleri ve C++11 tür güvenli `enum class` mimarisi

> **Yapı (struct):** Farklı veri tiplerindeki ilişkili değişkenleri tek bir bellek bloğunda birleştiren kullanıcı tanımlı bileşik veri tipi.

Hafta 6: Uygulamalı ve Mekatronik Hedefler

Laboratuvar oturumu ve gömülü sistem mimarisi kazanımları:

- Sensör ve Eyleyici Veri Paketleri:
  - Çok eksenli robot kol eklemlerinin ve IMU sensör verilerinin iç içe yapılarla modellenmesi
  - Büyük yapıların fonksiyonlara `const T&` ve `const T*` ile sıfır kopyalama maliyetiyle aktarılması
- Donanım Kaydedicisi (Register) Haritalama:
  - 8-bit ve 16-bit mikrodenetleyici kontrol/durum kaydedicilerinin bit alanları ve hibrit `union` ile yönetimi
- Seri Haberleşme ve CAN Bus Protokol Paketleme:
  - `#pragma pack(push, 1)` ile dolgu baytlarının kaldırılması ve 8-baytlık CAN Bus telemetri çerçevesi tasarımı
- Sonlu Durum Makinesi (FSM) Denetimi:
  - `enum class : uint8_t` tabanlı tip güvenli endüstriyel robot durum makinesi implementasyonu

CAN Bus (Controller Area Network): Otomotiv ve endüstriyel otomasyonda mikrodenetleyicilerin gürültüye dayanıklı haberleşmesini sağlayan seri veri yolu standardı.

Motivasyon: Dağınık Değişkenlerden Bileşik Veriye

Bir mekatronik sistemde motor veya sensör parametrelerini bağımsız değişkenlerle yönetmek ölçeklenemez:

- **Veri Bütünlüğü Kaybı:** Bir motorun kimliği, hızı, akımı ve hata durumu ayrı değişkenlerde tutulduğunda aralarındaki mantıksal bağ yazılımda görünmez hale gelir.
- **Fonksiyon Parametre Karmaşası:** Bir fonksiyona 8 ayrı motor parametresi göndermek hem okunabilirliği bozar hem de çağrı yığını (*call stack*) yönetimini zorlaştırır.
- **Çoklu Nesne Zorluğu:** Sisteme 1 motor yerine 6 eksenli bir robot kol eklendiğinde, bağımsız değişkenleri kopyalamak (`hiz1, hiz2, ...`) hataya açık kod tekrarı üretir.
- **Mühendislik Çözümü:** İlgili tüm nitelikleri `struct` çatısı altında birleştirerek tek bir atomik birim olarak taşımak ve işlemek.

> **Mühendislik İlkesi:** Fiziksel dünyada aynı nesneye ait olan büyüklükler (sıcaklık, basınç, zaman damgası), yazılım mimarisinde de tek bir veri yapısı içinde kapsüllenmelidir.

Yapı (`struct`) Tanımlama ve Sözdizimi

`struct` bildirimi derleyiciye yeni bir veri tipinin bellek şablonunu tanıtır:

```
1  // Dağınık yaklaşım (Önerilmez)
2  int    motorId   = 1;
3  double motorHizi = 1500.0;
4  bool   motorAktif = true;
5
6  // Bileşik yapı yaklaşımı (Modüler ve bütünleşik)
7  struct MotorSurucu {
8      int id;           // Motor kimlik numarası
9      double hizRpm;    // Anlık dönüş hızı (RPM)
10     double akimAmper; // Çekilen akım (A)
11     bool aktif;       // Sürücü çalışma durumu
12 }; // DİKKAT: Yapı tanımı sonunda noktalı virgül (;) zorunludur!
```

Kritik Kural: `struct MotorSurucu { ... };` satırı bellekte yer kaplamaz; yalnızca bir **tip şablonu (blueprint)** tanımlar. Fiziki bellek, bu tipten bir nesne oluşturulduğunda tahsis edilir.

Yapı Nesnesi Oluşturma ve Üye Erişimi (.)

Yapı türünden tanımlanan değişkenlerin üyelerine **nokta operatörü** (`.`) ile doğrudan erişilir:

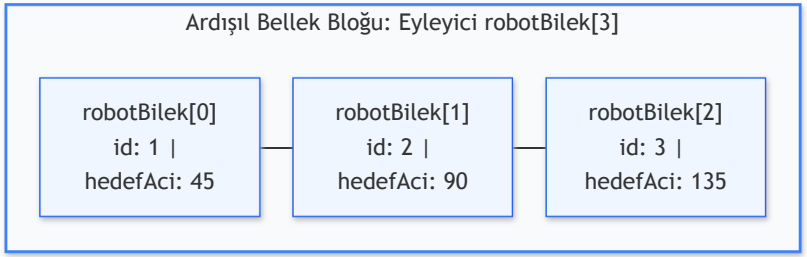
```
1  struct SensorVerisi {
2      int kanalNo;
3      double gerilim;
4      double sicaklik;
5  };
6
7  int main() {
8      // 1. Varsayılan tanımlama ve nokta (.) operatörü ile tekil atama
9      SensorVerisi s1;
10     s1.kanalNo  = 1;
11     s1.gerilim  = 3.32;
12     s1.sicaklik = 42.5;
13
14     // 2. Küme parantezi ile toplu ilklendirme (Aggregate Initialization)
15     SensorVerisi s2 = {2, 4.95, 28.1};
16     SensorVerisi s3{3, 1.80, 35.0}; // Modern C++11 tekbiçimli ilklendirme
17     return 0;
18 }
```

Toplu İlklendirme (Aggregate Initialization): Yapı üyelerinin bildirim sırasına uygun olarak süslü parantez `{}` içinde tek satırda ilklendirilmesi.

Yapı Dizileri (Array of Structs)

Çok kanallı sensör veya çok eksenli robot sistemlerinde aynı yapı tipinden oluşan diziler kullanılır:

- **Ardışıl Bellek Yerleşimi:** Yapı dizisinin her bir elemanı bellekte kesintisiz ve yan yana sıralanır (`dizi[0]` , `dizi[1]` , `dizi[2]`).
- **Erişim Sözdizimi:** Önce dizi indisi `[]` ile ilgili nesne seçilir, ardından nokta `.` operatörü ile üyesine ulaşılır: `robotKol[i].hedefAci` .
- **Döngü Entegrasyonu:** Aralık tabanlı `for` döngüsünde `const T&` kullanılarak tüm yapı elemanları kopyalanmadan hızla taranır.



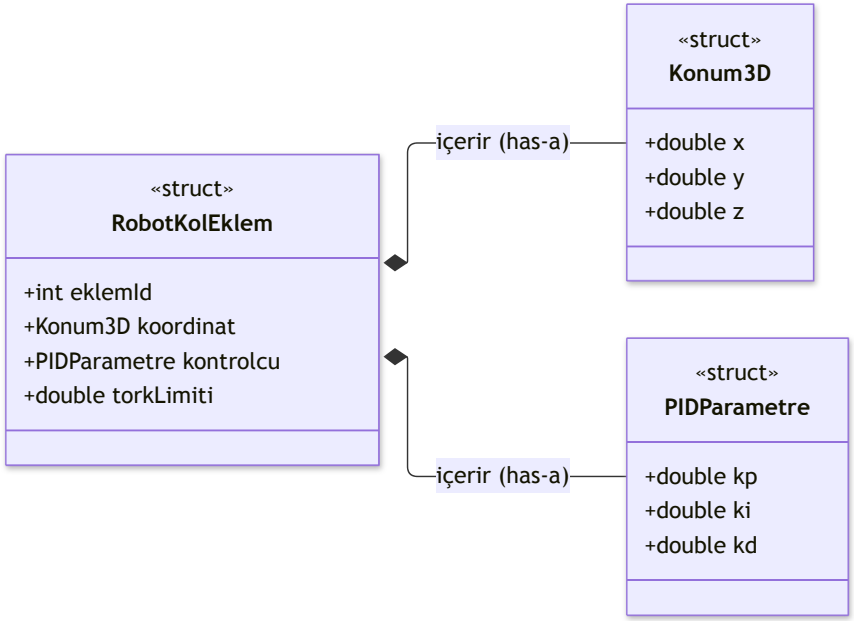
Yapı Dizileri: Çoklu Eyleyici Kontrolü

3 serbestlik dereceli bir robot bileğinin eklem verilerini yapı dizisi ile yöneten uygulama:

```
2
3   struct Eyleyici {
4       int id;
5       int hedefAci;    // Derece (0 - 180)
6       double torkNm;   // Newton-metre
7   };
8
9   int main() {
10      const int EKSEN_SAYISI = 3;
11      Eyleyici robotBilek[EKSEN_SAYISI] = {
12          {1, 45, 1.2}, {2, 90, 2.5}, {3, 135, 0.8}
13      };
14
15      for (const Eyleyici& motor : robotBilek) {
16          std::cout << "Eksen " << motor.id << " -> Aci: " << motor.hedefAci
17                    << " deg | Tork: " << motor.torkNm << " Nm\n";
18      }
19      return 0;
20  }
```

İç İçe Yapılar (Nested Structs) ve Kompozisyon

Karmaşık mekatronik sistemler, daha küçük alt yapıların birleştirilmesiyle (*kompozisyon*) modellenir:



- **Zincirleme Erişim:** İç yapı üyelerine kademeli nokta operatörüyle erişilir: `eklem1.koordinat.z`.
- **Yeniden Kullanılabilirlik:** `Konum3D` yapısı hem robot ekleminde hem de lidar hedef tespitinde ortak kullanılır.

İç İçe Yapılar: Robot Kol Eklem İmplementasyonu

```
2
3   struct Konum3D {
4       double x, y, z; // Milimetre (mm)
5   };
6
7   struct RobotKolEklem {
8       int eklemId;
9       Konum3D koordinat; // İç içe yapı üyesi
10      double torkLimiti;
11  };
12
13  int main() {
14      RobotKolEklem eklem1 = {101, {120.5, -45.0, 310.2}, 65.0};
15
16      std::cout << "Eklem ID : " << eklem1.eklemId << "\n";
17      std::cout << "Konum (X, Y, Z) : (" << eklem1.koordinat.x << ", "
18                << eklem1.koordinat.y << ", " << eklem1.koordinat.z << ") mm\n";
19      return 0;
20  }
```

Kompozisyon (Composition): Bir bileşik veri tipinin başka veri yapılarını üye değişken olarak kendi bünyesinde barındırması ("has-a" ilişkisi).

Yapı İşaretçileri ve Ok (`->`) Operatörü

Bir yapı nesnesine işaretçi (*pointer*) üzerinden erişirken **ok operatörü** (`->`) kullanılır:

- **Dolaylı Üye Erişimi:** `ptr->uye` ifadesi, işaretçinin gösterdiği nesneye ulaşıp üyesini okuyan `(*ptr).uye` ifadesinin birebir karşılığıdır.
- **Operatör Önceliği Riski:** Nokta (`.`) operatörü, dereferans (`*`) operatöründen daha yüksek önceliğe sahiptir; bu nedenle `*ptr.uye` yazımı derleme hatası üretir.
- **Bellek Verimliliği:** Büyük yapı nesneleri fonksiyonlara kopyalanmak yerine adresleri (`const Yapı*`) veya referansları (`const Yapı&`) üzerinden iletilir.

> **Sözdizimi Eşdeğerliği:**
> - Doğrudan nesne üzerinden: `paket.bataryaGerilimi`
> - İşaretçi üzerinden (Tercih edilen): `ptr->bataryaGerilimi`
> - Parantezli dereferans ile: `(*ptr).bataryaGerilimi`

Yapıların Fonksiyonlara Aktarımı (`const T*` ve `const T&`)

```
2
3   struct TelemetriPaketi {
4       unsigned int zamanDamgasiMs;
5       float bataryaGerilimi;
6       int hataKodu;
7   };
8
9   // İşaretçi ile aktarım: nullptr denetimi zorunludur
10  void telemetriYazdirPtr(const TelemetriPaketi* ptr) {
11      if (ptr == nullptr) return;
12      std::cout << "[PTR] Zaman: " << ptr->zamanDamgasiMs
13                << " ms | Batarya: " << ptr->bataryaGerilimi << " V\n";
14  }
15
16  // const Referans ile aktarım: Modern C++'ta tercih edilen güvenli yöntem
17  void telemetriYazdirRef(const TelemetriPaketi& ref) {
18      std::cout << "[REF] Zaman: " << ref.zamanDamgasiMs
19                << " ms | Batarya: " << ref.bataryaGerilimi << " V\n";
20  }
```

Dinamik Bellekte Yapı Tahsisi (`new` / `delete`)

Çalışma zamanında (*run-time*) oluşturulan yapı nesneleri Öbek (*Heap*) bellekte tahsis edilir:

```cpp
#include <iostream>

struct EnkoderVerisi {
    long pulsSayaci;
    int donusHiziRpm;
};

int main() {
    // Heap üzerinde dinamik yapı tahsisi ve ilklendirme
    EnkoderVerisi* enkPtr = new EnkoderVerisi{1048576L, 3000};

    std::cout << "Enkoder Puls: " << enkPtr->pulsSayaci
              << " | Hiz: " << enkPtr->donusHiziRpm << " RPM\n";

    delete enkPtr;    // Tahsis edilen öbek belleği sisteme iade edilir
    enkPtr = nullptr; // Sarkan gösterici (dangling pointer) önlenir
    return 0;
}
```

Enkoder (Rotary Encoder): Motor milinin açısal konumunu ve dönüş hızını optik veya manyetik darbeler (puls) üreterek ölçen mekatronik sensör.

`struct` Yapısı

Varsayılan erişim ve kalıtım `public` 'tir:

```
1  struct YapiNokta {
2      int x; // Varsayılan: public
3      int y;
4  };
5
6  int main() {
7      YapiNokta p;
8      p.x = 10; // Doğrudan erişilebilir
9  }
```

- Kullanım Amacı: Pasif veri paketleri, donanım mesaj çerçeveleri ve koordinat demetleri.

`class` Yapısı

Varsayılan erişim ve kalıtım `private` 'tır:

```
1  class SinifNokta {
2      int x; // Varsayılan: private
3      int y;
4  public:
5      void ayarla(int a, int b) {
6          x = a; y = b;
7      }
8  };
```

- Kullanım Amacı: Veri gizleme (*data hiding*) ve sınıf değişmezlerini (*invariants*) koruyan nesneler.

Yapılarda Kurucu Metot (Constructor)

C++ dilinde yapılar, nesne oluşturulduğu anda otomatik çalışan **kurucu metotlara** sahip olabilir:

```cpp
#include <iostream>

struct DCMotor {
    int id;
    int pwmPin;
    double maksAkim;

    // Üye ilklendirme listesi (Member Initializer List) ile kurucu
    DCMotor(int _id, int _pin, double _akim)
        : id(_id), pwmPin(_pin), maksAkim(_akim) {
        std::cout << "Motor " << id << " (Pin " << pwmPin << ") hazir.\n";
    }
};

int main() {
    DCMotor sagMotor(1, 9, 4.5); // Kurucu otomatik çağrılır
    return 0;
}
```

Üye İlklendirme Listesi (Member Initializer List): Kurucu gövdesi çalışmadan önce üyeleri doğrudan kendi bellek adreslerinde ilklendiren `: uye(deger)` yapısı.

Aşırı Yüklenmiş Kurucular (Constructor Overloading)

Bir yapı, farklı başlatma senaryolarını desteklemek üzere birden fazla kurucu barındırabilir:

```cpp
#include <iostream>

struct KalmanDurumu {
    double tahmin;
    double hataKovaryansi;

    // 1. Varsayılan (Default) Kurucu
    KalmanDurumu() : tahmin(0.0), hataKovaryansi(1.0) {}

    // 2. Parametreli Kurucu
    KalmanDurumu(double t, double p)
        : tahmin(t), hataKovaryansi(p) {}
};

int main() {
    KalmanDurumu k1;            // Varsayılan kurucu: (0.0, 1.0)
    KalmanDurumu k2(24.5, 0.05); // Parametreli kurucu: (24.5, 0.05)
    return 0;
}
```

Yapılarda Yıkıcı Metot (Destructor) ve RAII

Yapı nesnesi kapsam dışına çıktığında (*out of scope*) yıkıcı metot (`~YapiAdi`) otomatik çalışır:

```
1  #include <iostream>
2
3  struct SensorTamponu {
4      int* veriAlani;
5      int kapasite;
6
7      SensorTamponu(int cap) : kapasite(cap) {
8          veriAlani = new int[kapasite];
9          std::cout << kapasite << " elemanli dinamik tampon ayrildi.\n";
10     }
11
12     ~SensorTamponu() {
13         delete[] veriAlani; // Kapsam bitiminde otomatik bellek iadesi
14         std::cout << "Tampon bellegi guvenle serbest birakildi.\n";
15     }
16 };
```

RAII (Resource Acquisition Is Initialization): Dinamik bellek veya donanım portu gibi kaynakların kurucuda alınıp yıkıcıda otomatik iade edilmesi prensibi.

Birlikler (`union`) ve Ortak Bellek Paylaşımı

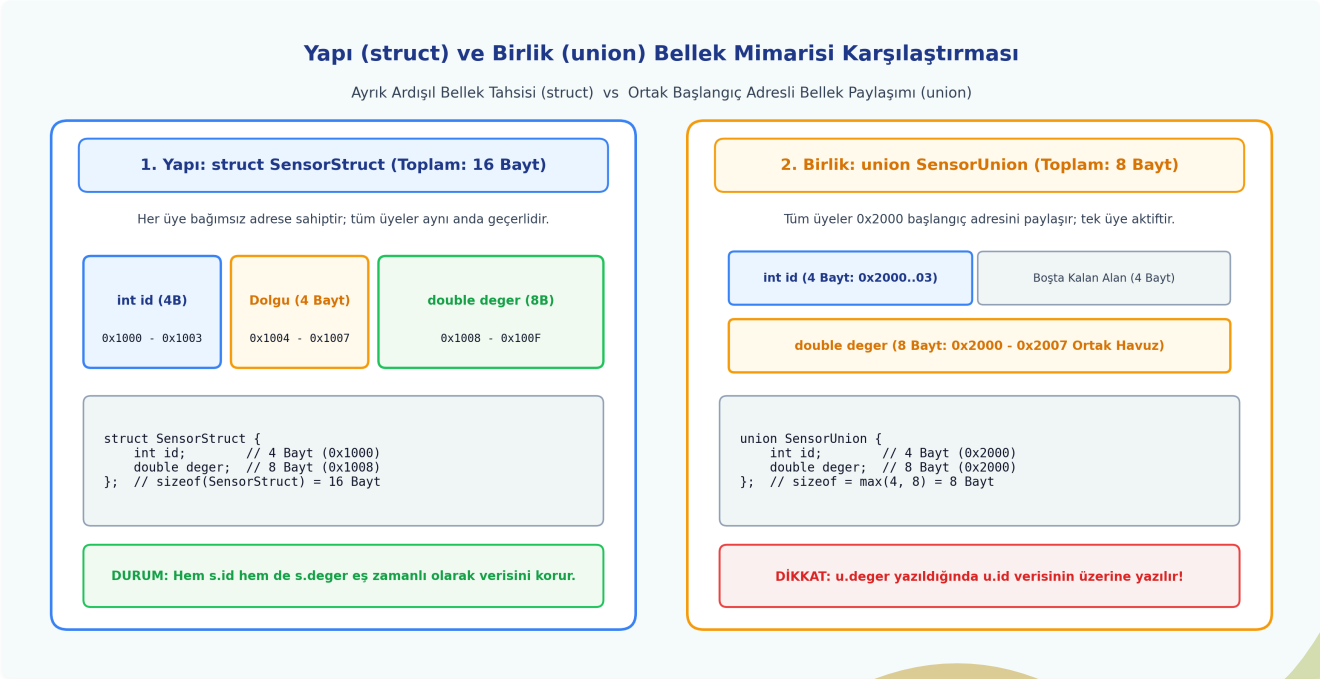
Birlik (`union`), tüm üyelerinin **aynı bellek başlangıç adresini** paylaştığı özel bir veri yapısıdır:

- **Ortak Bellek Havuzu:** Yapılarda (`struct`) her üye için ayrı bellek alanı tahsis edilirken, birliklerde tüm üyeler aynı fiziki bellek hücresinin üzerine yerleşir.
- **Boyut Kuralı:** Bir birliğin toplam bellek boyutu (`sizeof`), içindeki **en büyük üyenin boyutuna** (ve hizalama sınırına) eşittir.
- **Tek Aktif Üye İlkesi:** Herhangi bir anda birlikteki üyelerden **yalnızca biri** anlamlı veri tutabilir; yeni bir üyeye atama yapıldığında önceki üyenin bit dizilimi değişir.
- **Gömülü Sistem Kullanımı:** Kısıtlı RAM'e sahip mikrodenetleyicilerde ortak tampon paylaşımı ve donanım kaydedici erişimi için kullanılır.

> **Birlik (union):** Üyelerinin tamamı aynı bellek adresinden başlayan ve aynı anda yalnızca tek bir üyesi aktif olan düşük seviyeli veri yapısı.

`struct` ve `union` Bellek Mimarisi Karşılaştırması

Ayrık ardışıl bellek yerleşimi (`struct`) ile çakışan ortak bellek yerleşiminin (`union`) karşılaştırması:



- `struct SensorStruct` : `id` (`0x1000`) ve `deger` (`0x1008`) ayrı adreslerdedir; toplam boyut 16 Bayt'tır.

Birliklerde Tek Aktif Üye Davranışı

Birlik içinde bir üyeye değer yazıldığında ortak bellek alanı güncellenir:

```cpp
#include <iostream>

union DonanimKaydi {
    int   tamSayi;   // 4 Bayt
    float gerilim;   // 4 Bayt
}; // sizeof(DonanimKaydi) = 4 Bayt

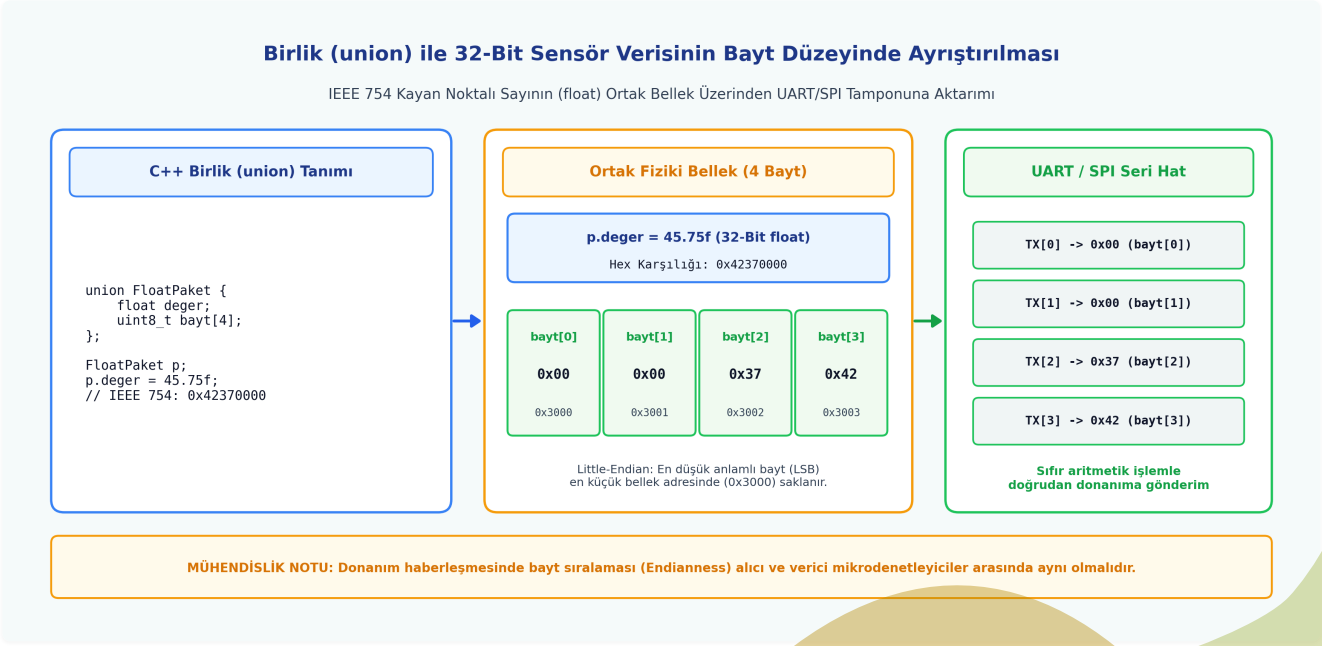
int main() {
    DonanimKaydi reg;
    reg.tamSayi = 100;
    std::cout << "Tam sayi aktif : " << reg.tamSayi << "\n"; // 100 (Geçerli)

    reg.gerilim = 3.3f; // Ortak 4 baytlık alanın üzerine IEEE 754 float yazıldı!
    std::cout << "Gerilim aktif  : " << reg.gerilim << "\n"; // 3.3 (Geçerli)
    std::cout << "Eski tam sayi  : " << reg.tamSayi << "\n"; // 1079194419 (Bozuldu!)
    return 0;
}
```

Uygulama Notu: `reg.gerilim = 3.3f` atamasından sonra `reg.tamSayi` okunduğunda görülen `1079194419` değeri, `3.3f` sayısının IEEE 754 ikili karşılığı olan `0x40533333` bit diziliminin tamsayı yorumudur.

Birlikler ile Bayt Düzeyinde Ayrıştırma (Type Punning)

Gömülü haberleşme protokollerinde (UART, SPI, I2C, CAN) çok baytlı veriler bayt dizisine dönüştürülür:



- Sıfır Hesaplama Yükü: 32-bit `float` sensör verisi, `uint8_t bayt[4]` dizisi ile aynı 4 baytlık belleği paylaşır.

32-Bit Sensör Verisinin Baytlara Ayrıştırılması

```cpp
#include <cstdint>

union FloatPaket {
    float   deger;     // 4 Bayt (32-bit IEEE 754)
    uint8_t bayt[4];   // 4 x 1 Bayt (İşaretsiz 8-bit dizi)
};

int main() {
    FloatPaket sensor;
    sensor.deger = 45.75f; // Bellekte: 0x42370000

    std::cout << "Float Deger: " << sensor.deger << "\nUART TX Baytlari: ";
    for (int i = 0; i < 4; ++i) {
        std::cout << "0x" << std::hex << std::uppercase << std::setw(2)
                  << std::setfill('0') << static_cast<int>(sensor.bayt[i]) << " ";
    }
    std::cout << std::dec << "\n";
    return 0;
}
```

Etiketli Birlik Tasarımı (Tagged Union)

Bir birliğin o anda hangi üyesinin geçerli olduğunu güvenle takip etmek için **Etiketli Birlik** deseni kullanılır:

- **Problem:** Çıplak bir `union` kendi içinde hangi üyenin aktif olduğu bilgisini taşımaz; yanlış üyenin okunması veri hatasına yol açar.
- **Mimari Çözüm:** Birlik yapısı, aktif veri tipini belirten bir `enum class` etiketi (*tag*) ile birlikte kapsayıcı bir `struct` içine yerleştirilir.
- **Anonim Birlik (Anonymous Union):** Yapı içinde isimsiz tanımlanan birlik üyelerine doğrudan dış yapı üzerinden erişilebilir.
- **Modern C++ Karşılığı:** C++17 standardındaki `std::variant` sınıfı, etiketli birlik mimarisinin tür güvenli şablon karşılığıdır.

> **Etiketli Birlik (Tagged / Discriminated Union):** Ortak bellek alanındaki aktif veri tipini bir durum etiketi (`enum`) ile birlikte saklayan güvenli yapı deseni.

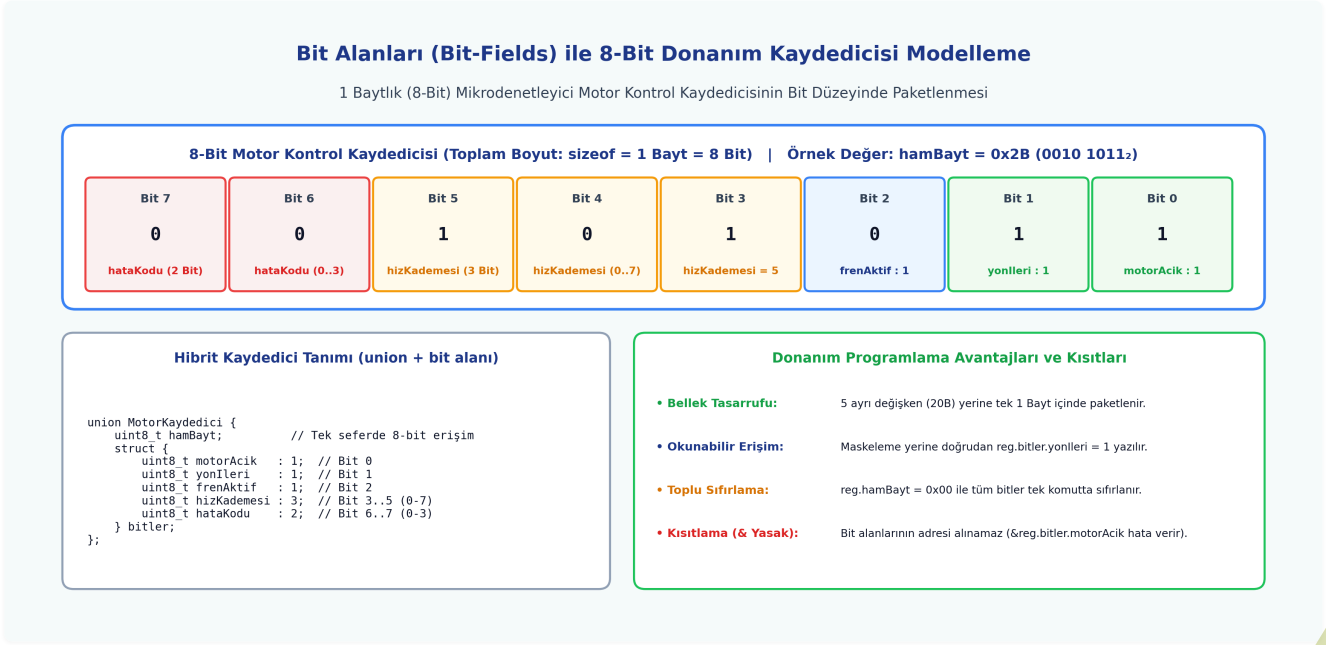
Etiketli Birlik C++ İmplementasyonu

```cpp
struct GuvenliTelemetri {
    VeriTipi aktifTip; // Hangi üyenin geçerli olduğunu tutan etiket
    union {
        float sicaklik;
        int   hizRpm;
        char  hataKarakteri;
    }; // Anonim birlik: doğrudan paket.sicaklik şeklinde erişilir
};

void paketiIsle(const GuvenliTelemetri& p) {
    switch (p.aktifTip) {
        case VeriTipi::SICAKLIK:
            std::cout << "Sicaklik : " << p.sicaklik << " C\n"; break;
        case VeriTipi::MOTOR_HIZI:
            std::cout << "Motor Hiz: " << p.hizRpm << " RPM\n"; break;
        case VeriTipi::HATA_KODU:
            std::cout << "Hata Kodu: " << p.hataKarakteri << "\n"; break;
    }
}
```

Bit Alanları (Bit-Fields) ve Donanım Modelleme

Mikrodenetleyici kaydedicilerinde (*registers*) her bir bit bağımsız bir donanım bayrağını temsil eder:



- **Bit Genişliği Tanımı:** `uint8_t uyeAdi : bitSayisi;` sözdizimi ile üyeye yalnızca ihtiyaç duyduğu kadar bit tahsis edilir.
- **8 Kat Bellek Tasarrufu:** 5 farklı motor durum bilgisi 5 ayrı değişkende (5–20 Bayt) tutulmak yerine tek 1

Bit Alanları ile 8-Bit Motor Kontrol Kaydedicisi

```
 3
 4  struct MotorKontrolKaydi {
 5      uint8_t motorAcik   : 1; // Bit 0   (0: Kapalı, 1: Açık)
 6      uint8_t yonIleri    : 1; // Bit 1   (0: Geri,   1: İleri)
 7      uint8_t frenAktif   : 1; // Bit 2   (0: Serbest,1: Frenli)
 8      uint8_t hizKademesi : 3; // Bit 3-5 (3 bit -> 0..7 arası 8 kademe)
 9      uint8_t hataKodu    : 2; // Bit 6-7 (2 bit -> 0..3 arası 4 durum)
10  };
11
12  int main() {
13      MotorKontrolKaydi drv = {1, 1, 0, 5, 0};
14
15      std::cout << "Kaydedici Boyutu : " << sizeof(MotorKontrolKaydi) << " Bayt\n";
16      std::cout << "Motor Yonu       : " << (drv.yonIleri ? "Ileri" : "Geri") << "\n";
17      std::cout << "Hiz Kademesi     : " << static_cast<int>(drv.hizKademesi) << " / 7\n";
18
19      // DİKKAT: 3-bitlik alana (maks 7) 9 yazılırsa taşma olur (9 % 8 = 1 yazılır!)
20      return 0;
21  }
```

Hibrit Kaydedici: Birlik ve Bit Alanı Sentezi

Donanım sürücülerinde kaydediciye hem **tekil bitler** halinde hem de **tek bir ham bayt** olarak erişilir:

```
6       struct {
7           uint8_t hazir      : 1; // Bit 0
8           uint8_t iletisimOk : 1; // Bit 1
9           uint8_t asiriAkim  : 1; // Bit 2
10          uint8_t asiriIsi   : 1; // Bit 3
11          uint8_t rezerve    : 4; // Bit 4-7
12      } bitler;
13  };
14
15  int main() {
16      DurumKaydedici reg;
17      reg.hamBayt = 0x05; // 0000 0101_2 -> hazir = 1, asiriAkim = 1
18
19      if (reg.bitler.asiriAkim) {
20          std::cout << "[ALARM] Asiri akim algilandi! Kaydedici sifirlaniyor.\n";
21          reg.hamBayt = 0x00; // Tüm bitleri tek satırda sıfırla
22      }
23      return 0;
24  }
```

Bit Alanlarında Kısıtlamalar ve Mimari Kurallar

Bit alanları donanım seviyesinde büyük kolaylık sağlasa da C++ standardında önemli kısıtlara sahiptir:

- **Adres Operatörü (`&`) Yasaktır:** Bellek adreslemesi bayt sınırlarında yapılır; tekil bir bitin adresi alınamaz (`&reg.bitler.hazir` derleme hatası verir).
- **İşaretçi ve Referans Bağlanamaz:** Bit alanlarına `uint8_t*` göstericisi veya `uint8_t&` referansı atanamaz.
- **Bit Dizilim Yönü (Implementation-Defined):** Bitlerin düşük bitten (*LSB*) mi yoksa yüksek bitten (*MSB*) mi yerleşeceği hedef işlemci mimarisine bağlıdır.
- **Değer Taşması (Bit Truncation):** k bitlik bir işaretsiz alana $[0, 2^k - 1]$ aralığı dışında değer atanırsa üst bitler sessizce kesilir.

> **Tasarım Notu:** Bit alanlarında işaret bitinden kaynaklanan sürprizleri önlemek için daima açıkça işaretsiz tamsayı tipleri (`uint8_t` , `uint16_t` , `uint32_t`) tercih edilmelidir.

Bellek Hizalama (Memory Alignment) Mekanizması

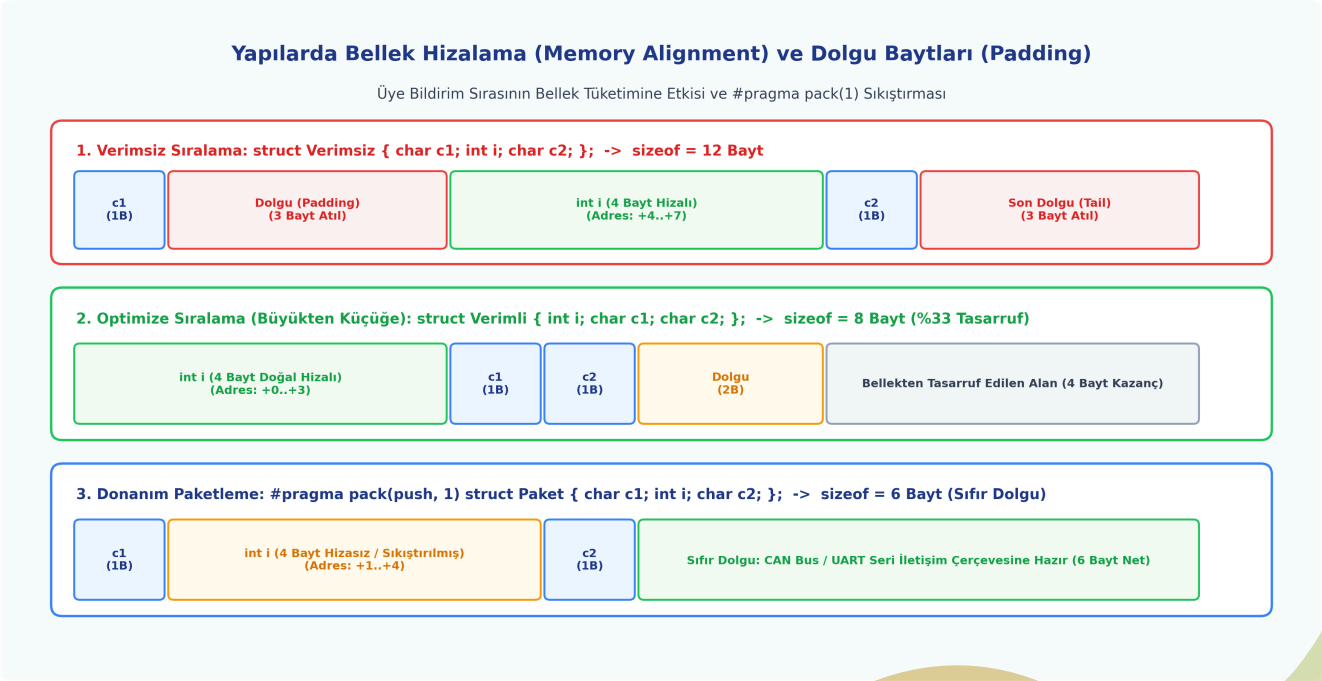
Modern 32-bit ve 64-bit işlemciler belleği tek tek baytlar halinde değil, 4 veya 8 baytlık kelimeler (*words*) halinde okur:

- **Doğal Hizalama Kuralı:** N bayt boyutundaki bir temel veri tipinin bellek başlangıç adresi, N sayısının tam katı olmalıdır:
  - `char` / `uint8_t` (1 Bayt) $\rightarrow$ Herhangi bir adrese yerleşebilir.
  - `int16_t` (2 Bayt) $\rightarrow$ Adresi 2'nin katı olmalıdır (`0x1000` , `0x1002` , ...).
  - `int32_t` / `float` (4 Bayt) $\rightarrow$ Adresi 4'ün katı olmalıdır (`0x1000` , `0x1004` , ...).
  - `double` (8 Bayt) $\rightarrow$ Adresi 8'in katı olmalıdır (`0x1000` , `0x1008` , ...).
- **Derleyici Müdahalesi:** Derleyici bu kuralı sağlamak için yapı üyelerinin arasına otomatik olarak **dolgu baytları** (*struct padding*) yerleştirir.

> **Doğal Hizalama (Natural Alignment):** İşlemcinin veriyi tek bellek çevriminde okuyabilmesi için veri adresinin veri boyutunun tam katı olması şartı.

Dolgu Baytları (Padding) ve `#pragma pack(1)`

Üye tanımlama sırasının bellek boyutuna etkisi ve donanım paketleme karşılaştırması:



- **Üye Sıralama Optimizasyonu:** Üyeleri büyükten küçüğe (`int` , `char` , `char`) sıralamak bellek tüketimini 12 Bayt'tan 8 Bayt'a düşürür.

Verimsiz Sıralama (12 Bayt)

Küçük ve büyük tiplerin karışık dizilmesi:

```
struct Verimsiz {
    char c1; // 1B + 3B Dolgu
    int  i;  // 4B (Hizalı)
    char c2; // 1B + 3B Son Dolgu
};
// sizeof(Verimsiz) == 12 Bayt
```

- **Bellek Kaybı:** 6 baytlık gerçek veri için **6 bayt (%50) atıl dolgu** harcanır.

Optimize Sıralama (8 Bayt)

Üyelerin büyükten küçüğe dizilmesi:

```
struct Verimli {
    int  i;  // 4B (Hizalı)
    char c1; // 1B
    char c2; // 1B + 2B Son Dolgu
};
// sizeof(Verimli) == 8 Bayt
```

- **Mühendislik Kazancı:** Hiçbir özel direktif kullanmadan **%33 bellek tasarrufu** sağlanır.

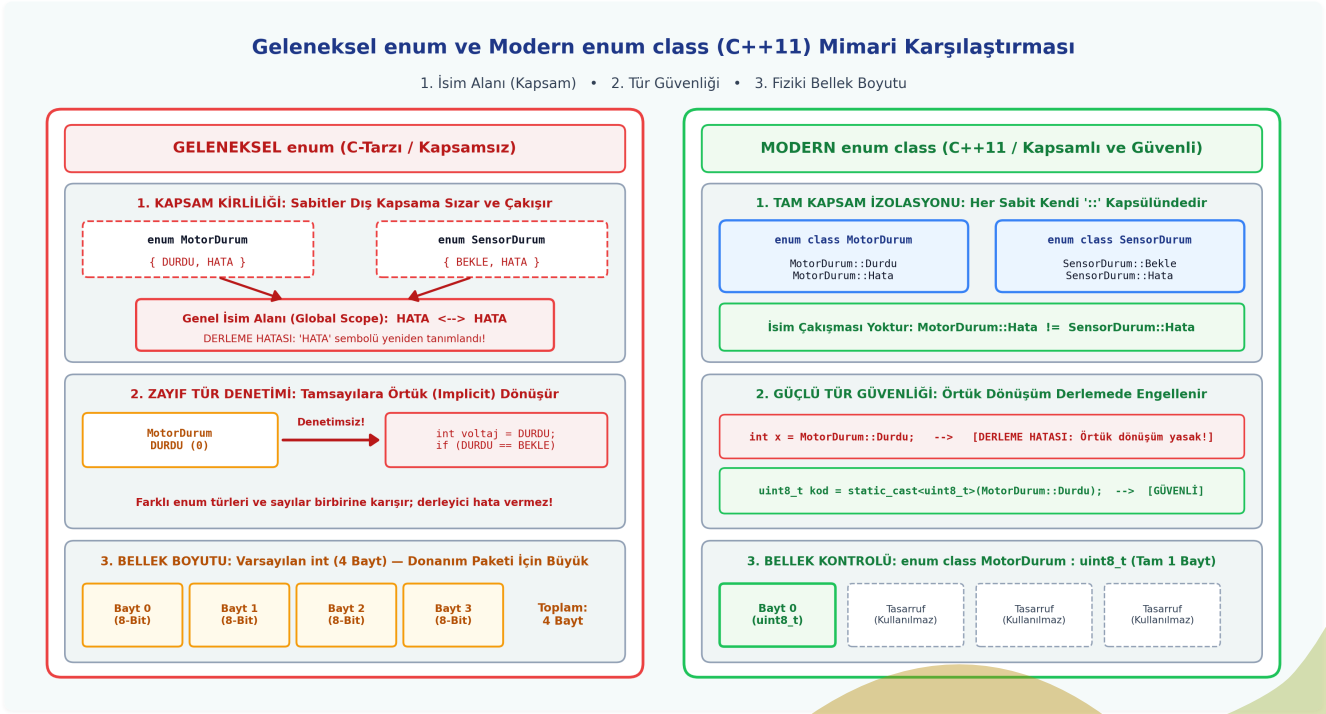
`#pragma pack` ve C++11 `alignof` / `alignas`

Haberleşme çerçevelerinde dolguyu kapatma ve donanım DMA tamponlarında özel hizalama:

```
3
4   #pragma pack(push, 1) // Derleyici dolgusunu 1-bayt sınırına çek (Dolgu yok)
5   struct SeriPaket {
6       uint8_t  baslik;   // 1 Bayt
7       uint32_t sensorId; // 4 Bayt (Dolgusuz hemen 1. bayttan başlar)
8       uint8_t  crc;      // 1 Bayt
9   }; // sizeof(SeriPaket) = 6 Bayt
10  #pragma pack(pop)     // Varsayılan derleyici hizalamasına geri dön
11
12  // DMA transferi için 16-bayt sınırına zorunlu hizalanmış yapı
13  struct alignas(16) DmaTamponu {
14      float kanallar[4]; // 16 Bayt
15  };
16
17  int main() {
18      std::cout << "SeriPaket Boyutu   : " << sizeof(SeriPaket) << " Bayt\n";
19      std::cout << "DmaTamponu Hizalama: " << alignof(DmaTamponu) << " Bayt\n";
20      return 0;
21  }
```

Geleneksel `enum` vs Modern `enum class` (C++11)

Klasik C numaralandırmalarının kapsam ve tip zafiyetleri ile modern `enum class` çözümü:



- Kapsam Güvenliği: `enum class` sabitlerine yalnızca `TipAdi::Sabit` (`MotorDurum::Calisiyor`) şeklinde erişilir, global isim alanı kirlenmez.

`enum class` ve Taban Bellek Türü (`: uint8_t`)

Gömülü sistem protokollerinde numaralandırmanın bellekte kaplayacağı bayt boyutu açıkça belirlenir:

```cpp
#include <iostream>
#include <cstdint>

// Taban türü 1 bayt (uint8_t) olarak sabitlenmiş kapsamlı numaralandırma
enum class HataKodu : uint8_t {
    HataYok       = 0x00,
    AsiriGerilim  = 0x01,
    AsiriSicaklik = 0x02,
    IletisimKopuk = 0x03
};

int main() {
    HataKodu durum = HataKodu::AsiriSicaklik;

    std::cout << "sizeof(HataKodu): " << sizeof(HataKodu) << " Bayt\n";
    std::cout << "Sayisal Kod     : " << static_cast<int>(durum) << "\n";
    return 0;
}
```

Taban Tür (Underlying Type): Numaralandırma sabitlerinin bellekte hangi tamsayı genişliğinde (`uint8_t` , `int16_t` , `int`) saklanacağını belirleyen bildirim.

Tür Takma Adları: `typedef` vs Modern `using` (C++11)

Karmaşık veri tiplerine ve fonksiyon işaretçilerine okunabilir takma adlar (*type aliases*) tanımlama:

```cpp
#include <iostream>
#include <cstdint>

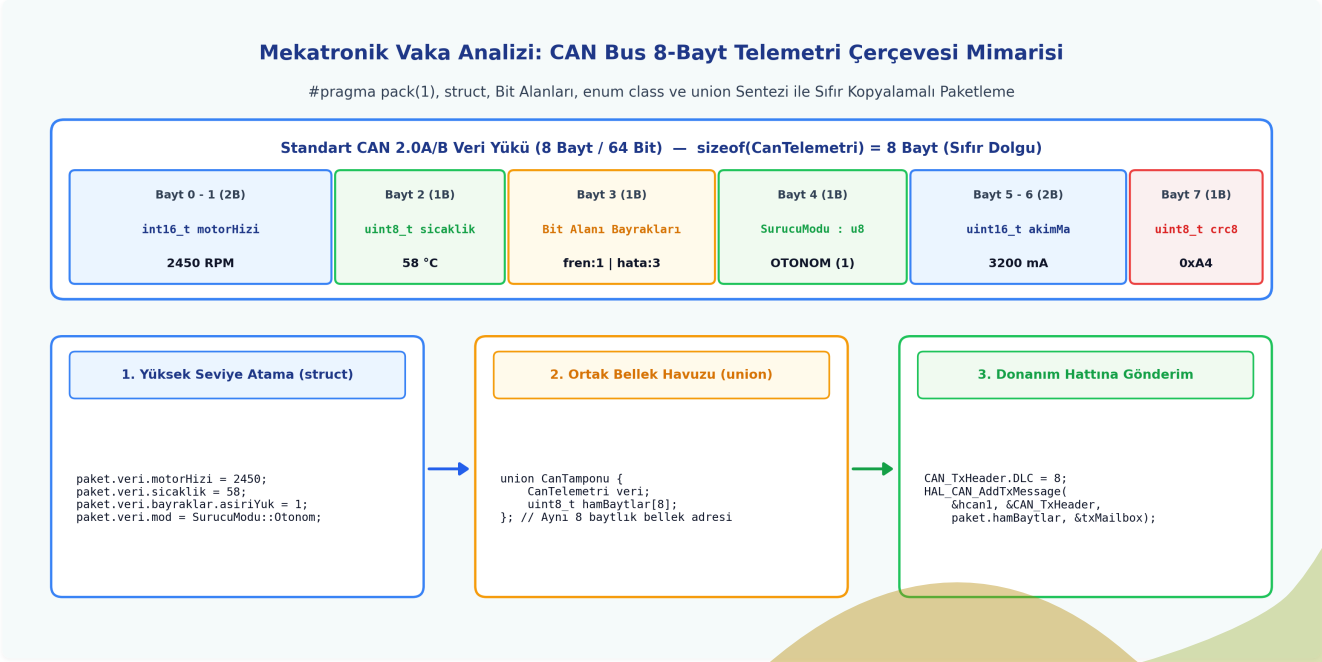
// 1. Geleneksel C-tarzı typedef bildirimi
typedef uint64_t Mikrosaniye_t;
typedef void (*KesmeIsleyici_old)(int);

// 2. Modern C++11 'using' bildirimi (Okunabilirlik ve şablon uyumu için önerilir)
using Mikrosaniye   = uint64_t;
using CanMesajId    = uint16_t;
using KesmeIsleyici = void (*)(int);

int main() {
    Mikrosaniye gecenSure = 250000ULL;
    CanMesajId  motorId   = 0x1A4;
    std::cout << "CAN ID: 0x" << std::hex << motorId << std::dec
              << " | Sure: " << gecenSure << " us\n";
    return 0;
}
```

Mekatronik Vaka Analizi 1: CAN Bus Telemetri Çerçevesi

Standart 8-baytlık CAN 2.0A/B veri çerçevesinin `struct`, bit alanları, `enum class` ve `union` ile tasarımı:



- **8-Bayt Donanım Sınırı:** Standart CAN çerçevesi en fazla 8 bayt taşır; `#pragma pack(push, 1)` ile dolgu

CAN Bus Çerçevesinin Veri Yapısı Tanımı

```
 7       int16_t    motorHiziRpm;     // Bayt 0-1 (2 Bayt: -32768 .. +32767 RPM)
 8       uint8_t    sicaklikDerece;   // Bayt 2   (1 Bayt: 0 .. 255 C)
 9       struct {
10           uint8_t motorAktif  : 1; // Bit 0
11           uint8_t frenDevrede : 1; // Bit 1
12           uint8_t asiriAkim   : 1; // Bit 2
13           uint8_t enkoderHata : 1; // Bit 3
14           uint8_t rezerve     : 4; // Bit 4-7
15       } bayraklar;                 // Bayt 3   (1 Bayt: 8 Bitlik durum alanı)
16       SurucuModu mod;              // Bayt 4   (1 Bayt: enum class)
17       uint16_t   akimMa;           // Bayt 5-6 (2 Bayt: 0 .. 65535 mA)
18       uint8_t    crc8;             // Bayt 7   (1 Bayt: Doğrulama özeti)
19   }; // Toplam Boyut: 2 + 1 + 1 + 1 + 2 + 1 = Tam 8 Bayt!
20   #pragma pack(pop)
21
22   union CanTamponu {
23       CanTelemetri veri;
24       uint8_t      hamBaytlar[8];
25   };
```

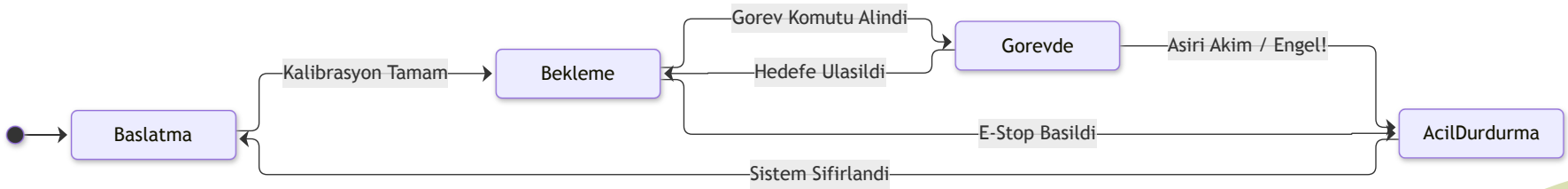
CAN Bus Veri Paketleme ve Çözümleme Kodu

```
    // 1. Yüksek seviyeli mühendislik birimleriyle yapı üyelerini doldur
    txPaket.veri.motorHiziRpm          = 2450;
    txPaket.veri.sicaklikDerece        = 58;
    txPaket.veri.bayraklar.motorAktif  = 1;
    txPaket.veri.bayraklar.frenDevrede = 0;
    txPaket.veri.bayraklar.asiriAkim   = 0;
    txPaket.veri.mod                   = SurucuModu::HizKontrol;
    txPaket.veri.akimMa                = 3200;
    txPaket.veri.crc8                  = 0xA4;

    // 2. Ortak bellek üzerinden 8 baytlık CAN çerçevesini seri hatta gönder
    std::cout << "CAN Cerceve Boyutu: " << sizeof(txPaket) << " Bayt\nCAN Hattina Giden: "
    for (int i = 0; i < 8; ++i) {
        std::cout << "0x" << std::hex << std::uppercase << std::setw(2)
                  << std::setfill('0') << static_cast<int>(txPaket.hamBaytlar[i]) << " ";
    }
    std::cout << std::dec << "\n";
    return 0;
}
```

Mekatronik Vaka Analizi 2: Robot Durum Makinesi (FSM)

Endüstriyel bir mobil robotun çalışma modları `enum class` ve `struct` birleşimiyle yönetilir:



- **Durum İzolasyonu:** `RobotDurum` numaralandırması geçersiz durum geçişlerini derleme zamanında önler.
- **Bağlam Yapısı (** `RobotBaglami` **):** Robotun anlık durumu, hızı ve hata bayrakları tek bir `struct` içinde taşınır.

Robot Durum Makinesi C++ İmplementasyonu

```
};

void durumGuncelle(RobotBaglami& robot) {
    if (robot.engelAlgilandi) {
        robot.durum = RobotDurum::AcilDurdurma;
        robot.hedefHizMsn = 0.0;
    }
    switch (robot.durum) {
        case RobotDurum::Baslatma:
            std::cout << "[FSM] Sensor kalibrasyonu yapiliyor...\n";
            robot.durum = RobotDurum::Bekleme; break;
        case RobotDurum::Bekleme:
            std::cout << "[FSM] Sistem hazir, gorev bekleniyor.\n"; break;
        case RobotDurum::Gorevde:
            std::cout << "[FSM] Hareket suruyor. Hiz: " << robot.hedefHizMsn << " m/s\n";
        case RobotDurum::AcilDurdurma:
            std::cout << "[E-STOP] Frenler kilitlendi! Hiz: 0 m/s\n"; break;
    }
}
```

Laboratuvar Alıştırması 1: IMU 6-Eksen Sensör Paketi

Problem Tanımı: Bir otonom insansız hava aracının (İHA) 6-eksenli IMU (*Inertial Measurement Unit*) sensöründen okunan 3-eksen ivmeölçer (a_x, a_y, a_z) ve 3-eksen jiroskop (g_x, g_y, g_z) verilerini modelleyen iç içe yapıyı ve sıfır-ofset kalibrasyon fonksiyonunu tasarlayınız.

- **Gereksinimler:**
  1. `Vektor3B` yapısı `double x, y, z;` üyelerini ve varsayılan kurucuyu içermelidir.
  2. `ImuPaketi` yapısı; paket numarası, `Vektor3B ivme` $(\mathrm{m/s^2})$ ve `Vektor3B jiroskop` $(\mathrm{deg/s})$ üyelerini barındırmalıdır.
  3. `kalibreEt(ImuPaketi& paket, const Vektor3B& jiroOfset)` fonksiyonu referans yoluyla jiroskop sapmasını düzeltmelidir.
  4. `raporla(const ImuPaketi& paket)` fonksiyonu veriyi kopyalamadan ekrana yazdırmalıdır.

> **IMU (Ataletsel Ölçüm Birimi):** İvmeölçer ve jiroskop sensörlerini birleştirerek aracın doğrusal ivmesini ve açısal hızını ölçen mekatronik algılayıcı.

Alıştırma 1: IMU Kalibrasyon C++ Çözümü

```cpp
    Vektor3B jiroskop; // deg/s
};

void jiroskopKalibreEt(ImuPaketi& p, const Vektor3B& ofset) {
    p.jiroskop.x -= ofset.x;
    p.jiroskop.y -= ofset.y;
    p.jiroskop.z -= ofset.z;
}

int main() {
    ImuPaketi imu = {1, {0.02, -0.01, 9.81}, {1.25, -0.80, 0.45}};
    Vektor3B  sapmaOfseti(1.20, -0.80, 0.40);

    jiroskopKalibreEt(imu, sapmaOfseti);

    std::cout << "Kalibre Jiroskop (deg/s): X=" << imu.jiroskop.x
              << " Y=" << imu.jiroskop.y << " Z=" << imu.jiroskop.z << "\n";
    return 0;
}
```

Laboratuvar Alıştırması 2: 16-Bit ADC Donanım Kaydedicisi

Problem Tanımı: Bir mikrodenetleyicinin 16-bitlik (2 Bayt) ADC sonuç kaydedicisi şu bit yapısına sahiptir:

- **Bit 0–11 (12 Bit):** Ham ADC ölçüm değeri ($0 \ldots 4095$)
- **Bit 12–14 (3 Bit):** Kanal numarası ($0 \ldots 7$)
- **Bit 15 (1 Bit):** Dönüşüm tamamlandı bayrağı (`hazir` : 0 veya 1)

Bu kaydediciyi hem `uint16_t hamVeri` hem de bit alanları olarak okuyabilen hibrit bir `union` ile modelleyiniz ve `0xB800` / `0xB400` değerlerini çözümleyiniz.

> **Mühendislik İpucu:** 12-bitlik alanı tutabilmek için bit alanı üyesinin taban tipi en az 16-bitlik `uint16_t` olarak seçilmelidir (`uint8_t` en fazla 8 bit tutabilir).

Alıştırma 2: 16-Bit ADC Kaydedicisi C++ Çözümü

```cpp
        uint16_t adcDegeri : 12; // Bit 0..11  (0 - 4095)
        uint16_t kanalNo   : 3;  // Bit 12..14 (0 - 7)
        uint16_t hazir     : 1;  // Bit 15     (0 veya 1)
    } alanlar;
};

int main() {
    AdcKaydedici16 adc;
    adc.alanlar.adcDegeri = 2048; // Yarı ölçek (1.65V)
    adc.alanlar.kanalNo   = 3;    // Kanal 3
    adc.alanlar.hazir     = 1;    // Ölçüm hazır

    std::cout << "Kaydedici Boyutu : " << sizeof(AdcKaydedici16) << " Bayt\n";
    std::cout << "16-Bit Hex Deger : 0x" << std::hex << std::uppercase << adc.hamKelime <<
    std::cout << "Kanal: " << adc.alanlar.kanalNo
              << " | ADC: " << adc.alanlar.adcDegeri
              << " | Gerilim: " << (adc.alanlar.adcDegeri * 3.3 / 4095.0) << " V\n";
    return 0;
}
```

Tasarım ve Uygulama Notları (Mühendislik İlkeleri)

Düşük seviyeli C++ veri yapılarında uyulması gereken yazılım mühendisliği kuralları:

- **Tasarım Notu 1 (Parametre Aktarımı):** Boyutu 8 baytı aşan tüm `struct` nesneleri fonksiyonlara değerle değil, `const Yapı&` referansı ile aktarılmalıdır.
- **Tasarım Notu 2 (Bellek Hizalama):** Yapı üyelerini bellek boyutlarına göre **büyükten küçüğe** (`double` $\rightarrow$ `int` $\rightarrow$ `short` $\rightarrow$ `char`) sıralamak derleyici dolgusunu (*padding*) en aza indirir.
- **Uygulama Notu 1 (`#pragma pack` Sınırı):** `#pragma pack(push, 1)` yalnızca donanım/ağ protokol çerçeveleri için kullanılmalı, sonrasında mutlaka `#pragma pack(pop)` ile normal hizalamaya dönülmelidir.
- **Uygulama Notu 2 (`enum class` Tercihi):** Yeni C++ projelerinde isim çakışmalarını ve örtük dönüşüm hatalarını önlemek için klasik `enum` yerine daima `enum class` kullanılmalıdır.

Hafta 6: Özet ve OOP Sınıflarına Köprü

Bu haftanın kazanımları, önümüzdeki hafta başlayacak Nesne Yönelimli Programlama sınıflarının temelini oluşturur:

- **Yapılardan Sınıflara Geçiş:** C++'ta `struct`, hem veri üyeleri hem de kurucu/yıkıcı metotlar barındırabilen tam yetenekli bir kullanıcı tanımlı türdür.
- **Kapsülleme İhtiyacı:** `struct` içindeki üyeler varsayılan olarak herkese açıktır (`public`). Veriyi dış müdahalelerden korumak ve geçerlilik denetimi yapmak için 7. haftada `class` ve `private` kapsülleme mimarisine geçeceğiz.
- **Donanım Seviyesi Hakimiyet:** `union`, bit alanları ve `#pragma pack`, C++ dilini gömülü sistemlerde ve mekatronikte vazgeçilmez kılan düşük seviyeli bellek araçlarıdır.

Gelecek Hafta: Hafta 7 Önizleme

Hafta 7: OOP'ye Giriş — Sınıflar, Nesneler ve Yaşam Döngüsü

- **Sınıf (`class`) ve Nesne (`object`) Paradigması:** Durum (*state*) ve davranış (*behavior*) bütünlüğü.
- **Erişim Belirleyicileri:** `public` , `private` ve `protected` görünürlük kuralları ve veri gizleme.
- **Nesne Yaşam Döngüsü:** Varsayılan, parametreli ve aşırı yüklenmiş kurucular (*constructors*) ile yıkıcılar (*destructors*).
- **Modern C++ Güvenceleri:** Üye ilklendirme listesi zorunlulukları ve `explicit` anahtar kelimesi ile örtük dönüşümlerin engellenmesi.
- **Arkadaş (`friend`) Mekanizması:** `friend` fonksiyonlar ve sınıflar ile kontrollü erişim.

Kaynaklar ve İletişim

- Ders Deposu: github.com/marmara-mekatronik/MRM3049
- Kaynak Kitaplar:
  - Deitel & Deitel, *C++ How to Program* (Bölüm 22: Bits, Characters, C Strings and structs)
  - D.S. Malik, *Data Structures Using C++* (struct ve Soyut Veri Tipleri Bölümü)
- **İletişim:** Dr. Hüseyin Yüce (Marmara Üniversitesi Teknoloji Fakültesi / Mekatronik Mühendisliği)

Nesne Yönelimli Programlama • Hafta 6: Yapılar, Birlikler ve Numaralandırmalar